flash-comm: Self-Synchronizing Collective Communication

Efficient Transport for Distributed LLM Decoding

Carrie Wang ScitiX.AI
Jack Li ScitiX.AI
Jason Zhao ScitiX.AI

arXiv:2611.06634v1 [cs.DC] 4 Nov 2026

Abstract

Collective communication, which synchronizes data across a group of GPUs, lies on the critical path of decoding in distributed large language model (LLM) serving. Despite advances in GPU interconnects and communication libraries, these operations still incur substantial overhead. Our measurements reveal that inter-GPU barriers, which are used to coordinate data readiness, can cost more than the data transfer itself, yet this overhead is often overlooked in existing communication optimizations.

We present flash-comm, a communication library that achieves efficient transport for LLM decoding by *fusing readiness synchronization with data movement*. flash-comm introduces *self-synchronizing transfers* in which each data block carries evidence of its readiness, allowing receivers to directly validate the blocks they need without separate inter-GPU barriers. Based on this principle, a cost-guided protocol selector adapts the readiness encoding to message size and the number of participating GPUs, balancing synchronization overhead against additional data-transfer cost. flash-comm further *decouples transport order from reduction order* to preserve result consistency across GPUs under asynchronous data arrivals.

On a server with eight NVIDIA B300 GPUs, flash-comm reduces all-reduce latency by a median of 28.8% and up to 43.7% compared with custom AR v2 (the state-of-the-art all-reduce implementation). In end-to-end decoding, flash-comm reduces time per output token (TPOT) by 1.7–6.8%. Note that all-reduce accounts for 11.4% of the baseline decoding time, which bounds the latency reduction possible from optimizing this operation alone. The same mechanism also supports standalone all-gather, reducing latency by a median of 23.8% and up to 58.9% across eight B300 GPUs.

1 Introduction

Distributed large language model (LLM) serving partitions computation across GPUs, which must exchange intermediate results before dependent computation can proceed. During decoding, these exchanges take the form of collective operations that recur across layers and generated tokens, so even microsecond-scale overhead accumulates on the critical path. Faster interconnects and optimized communication libraries have reduced this cost, yet it remains substantial: in a profiled workload, all-reduce alone accounts for 11.4% of the baseline decoding time. Further reducing communication overhead is therefore important for lowering time per output token (TPOT).

To understand where this overhead comes from, we examine the synchronization needed to exchange data safely. Inter-GPU barriers establish when data is ready to read and when shared buffers can be reused, but incur fixed coordination costs even for short transfers. On B300, for example, a cross-GPU barrier costs 4.33 μs, compared with 3.33 μs for an entire 1 KB all-reduce. To eliminate this barrier overhead, we present flash-comm, a communication library for LLM decoding that *fuses readiness synchronization with data movement*. Its *self-synchronizing transfers* make each data block carry evidence of its readiness, so receivers can directly validate the blocks they need. This allows communication to progress without separate inter-GPU barriers while preserving data dependencies and safe buffer reuse.

Figure 1 summarizes how these ideas fit together. flash-comm replaces group-wide readiness coordination with block-level readiness evidence, selects the readiness encoding and communication path according to their fixed and per-byte costs, and applies a deterministic reduction order when asynchronously delivered values are combined.

Realizing this design efficiently requires choosing how to encode readiness. Encoding it in data values avoids extra transmitted bytes but requires checking and resetting the buffers, whereas explicit sequence numbers simplify these operations at the cost of additional traffic. The balance depends on the communication workload: extra bytes matter less for short messages, but become more costly as transfer volume grows. To account for this tradeoff, flash-comm uses a *cost-guided protocol selector* that separates fixed overhead from per-byte cost. Measurements calibrate the switching boundaries for different message sizes and GPU configurations.

Beyond this performance tradeoff, allowing transfers to progress asynchronously raises a correctness concern for reductions. Because floating-point addition is not associative, GPUs that accumulate contributions in arrival order can produce different results from the same inputs. To preserve consistency, flash-comm *decouples transport order from reduction order*. When GPUs compute the same output independently, they accept contributions in any arrival order

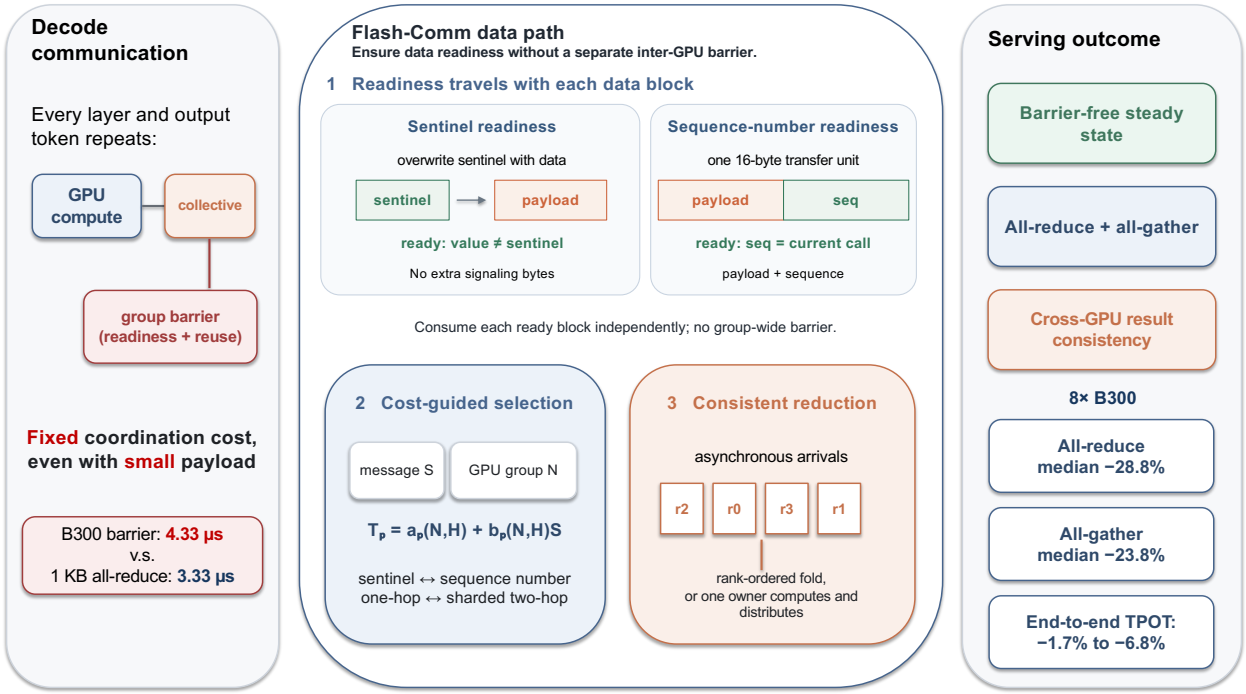


Figure 1: Overview of flash-comm.

but accumulate them in a common rank order. For sharded reductions, a single GPU computes each output block and distributes the same result to the others. In both cases, consistency does not require serializing data arrival.

We evaluate these mechanisms through collective communication benchmarks and end-to-end decoding workloads. On a server with eight NVIDIA B300 GPUs, flash-comm reduces all-reduce latency by a median of 28.8% and up to 43.7% compared with custom AR v2. Applying the same transfer mechanism to standalone all-gather reduces latency by a median of 23.8% and up to 58.9% across eight B300 GPUs. These communication improvements also translate to lower token generation latency: in end-to-end decoding, flash-comm reduces TPOT by 1.7–6.8%.

2 Background and Motivation

2.1 Collective Communication in LLM Decoding

Distributed LLM serving partitions model execution across multiple GPUs. Each GPU computes only part of an operator, so the devices must synchronize their intermediate data before dependent computation can proceed. These data exchanges are expressed as collective operations. All-reduce combines partial results and returns the result to every GPU; all-gather assembles distributed pieces on every GPU; and all-to-all sends each piece to its designated destination. Together, these operations support common serving strategies such as tensor parallelism and expert parallelism.

Communication is particularly visible during autoregressive decoding. Each decoding step produces only the next token or a short speculative sequence, but repeats the same communication dependencies across layers and generated tokens. A few microseconds added to one collective are therefore paid many times along the token-generation path. The message size varies with the model, batch size, data type, and parallel execution strategy, spanning transfers where fixed protocol overhead dominates and transfers where data movement dominates. In a representative profiled workload, all-reduce takes 1.68 ms of a 14.73 ms decoding iteration, or 11.4% of the baseline time. Collective communication therefore remains a meaningful optimization target even in an optimized serving stack.

2.2 Data Readiness and Buffer Reuse

Fast links do not by themselves tell a receiver when remote data is safe to consume. A GPU may enter its communication kernel while a peer is still producing its input, or before that peer's writes are visible across the interconnect. Reading the shared buffer at that point can return data from the previous invocation or an incompletely published value. A communication protocol must therefore establish that the producer has finished writing the required data and that the consumer can observe those writes.

The reverse dependency governs buffer reuse. A producer cannot overwrite a shared region for the next invocation while another GPU may still be reading the current value. Multi-stage collectives add a dependency between phases. For example, an all-reduce implemented as reduce-scatter followed by all-gather cannot distribute a reduced block until that block is complete. Correct execution thus requires three related guarantees: data is published before it is consumed, one phase completes before its dependent phase starts, and storage is reused only after the relevant consumers are done.

Table 1: Net cross-GPU barrier latency is comparable to the latency of a complete 1 KB all-reduce. Both are measured under CUDA Graph execution.

Machine	Net barrier latency	1 KB all-reduce latency
H100	2.66 μs	2.54 μs
H200	1.87 μs	2.46 μs
B300	4.33 μs	3.33 μs

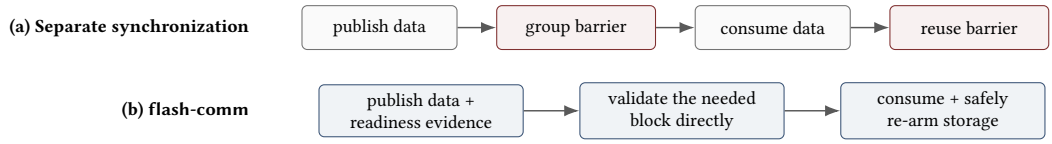


Figure 2: Readiness travels with each data block. Unlike a barrier-based protocol, flash-comm lets each receiver validate and consume a block as soon as it is ready, then safely reuse its storage.

Inter-GPU barriers are a common way to establish these relationships for the whole group. Together with the required memory-ordering operations, a barrier places every participant on the same side of a protocol boundary before dependent reads or buffer reuse. This group-wide confirmation is sufficient, but it is stronger than the immediate dependency of a consumer, which needs to know only whether its particular data block is ready. If each block carries reliable evidence of readiness, a receiver can validate the blocks it needs directly. Buffer reuse can likewise follow evidence that the relevant value has been consumed. This observation motivates the self-synchronizing transfers in flash-comm: preserve the data dependencies, but establish them through the data path instead of a separate group-wide confirmation step.

2.3 The Cost of Separate Synchronization

A barrier carries little payload, yet it still requires participants to publish and observe state across GPUs. Its latency is therefore largely a fixed protocol cost and does not shrink with the message being exchanged. We measure this cost under CUDA Graph execution using a flat cross-GPU barrier, then subtract the empty-kernel floor to isolate the inter-GPU component. Table 1 compares that net cost with the latency of a complete 1 KB all-reduce on the same machine. As shown in the table, the standalone barrier costs more than the entire small-message all-reduce on B300.

These measurements show that establishing readiness can cost as much as the small collective itself. A receiver fundamentally needs to wait only for the data on which it depends, whereas a barrier adds a group-wide coordination round to this local dependency. Block-level readiness instead lets the receiver proceed as soon as its required data arrives. flash-comm follows this approach and couples safe consumption and buffer reuse to the same readiness evidence. The next sections present the encodings and data paths that realize this principle efficiently.

3 Flash-Comm Design

flash-comm is built around one principle: readiness synchronization should travel with the data whose readiness it establishes. This turns synchronization from a group-wide protocol phase into a property that a receiver checks for each data block. Two further decisions make the principle practical. First, the readiness encoding and communication path are selected according to their fixed and per-byte costs. Second, reduction follows a deterministic evaluation order even when transport is asynchronous. This section presents these three parts of the design.

3.1 Self-Synchronizing Transfers

Figure 2 contrasts a barrier-based transfer with flash-comm. In a barrier-based protocol, producers publish data and then enter a separate group-wide synchronization step. Consumers wait for that step before reading, even when the particular block they need is already available. A second coordination point may be needed before buffers can be reused. In flash-comm, the published block carries its own readiness evidence. A consumer polls the block, consumes it as soon as it is ready, and re-arms or advances the storage according to that block's consumption rule. Readiness evidence directly orders consumption and storage reuse at block granularity. flash-comm provides two mechanisms for carrying this evidence: sentinel readiness and sequence-number readiness.

Sentinel readiness. The primary encoding reserves a bit pattern as a sentinel and pre-fills receive storage with it. A producer publishes a data pack by overwriting the sentinel. Observing a non-sentinel pack therefore proves that the pack has arrived, without adding a flag to the transmitted payload. After the sole consumer has captured the value, it restores the sentinel where the buffer-reuse rule permits.

All fp32 and bf16 bit patterns are valid inputs, so a real value can collide with the sentinel. flash-comm prevents such a collision from looking permanently unready by replacing that exact pattern with a canonical quiet NaN before publication. The same sanitization is applied to partial sums because a reduction can create the sentinel pattern even when no individual input has it. Legitimate NaN and infinity values create a second ambiguity in a switch-side reduction:

Table 2: The two readiness encodings trade fixed protocol work for transferred bytes. The selector chooses between them according to the communication workload and measured hardware costs.

Choice	Reduces	Adds	Favoured when
Sentinel readiness	transmitted signaling bytes	checking and re-arming	traffic matters
Sequence numbers	checking and re-arming	2× transferred bytes	fixed cost dominates

a non-finite result may mean either "an input is still the sentinel" or "the true result is non-finite." After bounded polling, flash-comm verifies each source explicitly and completes the reduction once every source has published.

Sequence-number readiness. An alternative encoding places data and a call sequence number in the same 16-byte transfer unit. A receiver accepts the data only when the sequence number matches the current invocation. This avoids sentinel tests and buffer re-arming, but only half of each transfer unit holds payload, doubling the traffic for the same useful data.

Both encodings operate over a set of rotating buffer slots. The slot is chosen by a device-side call counter so that CUDA Graph replay does not depend on a host parameter frozen during capture. Slot rotation delays reuse, while the readiness protocol establishes whether the value in a slot belongs to the current operation. Together, they separate successive invocations while keeping every rank aligned on the same collective sequence.

3.2 Cost-Guided Protocol Selection

No readiness encoding or communication path is best for every workload. Sentinels carry no additional payload but require tests and re-arming stores; sequence numbers reduce that fixed work but increase the number of bytes. Likewise, a one-hop all-reduce has one communication phase but sends a full contribution toward every peer, whereas a sharded two-hop path adds a phase and reduces the data volume handled by each GPU.

Within a stable protocol band, flash-comm models latency as

$$T_p(S; N, H) = a_p(N, H) + b_p(N, H)S, \tag{1}$$

where S is message size, N is the number of participating GPUs, and H identifies the hardware. The intercept a_p includes launch, readiness handling, and protocol round trips; the slope b_p captures per-byte transfer and memory cost. We calibrate the fixed and per-byte costs for each target platform. The model exposes the central tradeoff: reducing fixed work may add bytes, while reducing bytes may add a phase. Table 2 summarizes this tradeoff for the two readiness encodings.

For a unicast byte account, a one-hop exchange moves approximately $(N-1)S$ per GPU, while reduce-scatter plus all-gather moves $2(N-1)S/N$. At $N = 8$, these are $7S$ and $1.75S$; at $N = 4$, they are $3S$ and $1.5S$. At $N = 2$, both move S, so the second phase has no byte saving to repay its dependency cost. The two-GPU shipping policy consequently uses one-hop paths throughout its supported range.

For two candidate protocols, the crossing of their locally fitted cost lines identifies the switching region. flash-comm benchmarks the neighbouring sizes and places the final boundary on a measured point, accounting for discrete changes in grid shape, pipeline depth, and bandwidth saturation. At runtime, a constant-time lookup indexed by GPU architecture, group size, and message size returns the selected policy entry.

In the resulting policy, sequence-number readiness is used on Blackwell for two-GPU messages up to 128 KB. Sentinel readiness is used for the remaining shipping configurations, including the one-hop data exchange and the boundary between reduce-scatter and all-gather in the sharded path.

3.3 Reduction Consistency under Asynchronous Arrival

Self-synchronizing transfers allow data blocks to arrive in different orders. That freedom is harmless for data movement, but not automatically for floating-point reduction. Floating-point addition is not associative, so two GPUs that fold the same contributions in different arrival orders can produce different bit patterns. A tolerance check against a high-precision reference may accept both results, even though a subsequent argmax, threshold, or routing decision can diverge across GPUs.

flash-comm separates the order in which values arrive from the order in which they are evaluated. For replicated one-hop reductions, each contribution is first stored in a slot identified by its source rank. Once all required slots are ready, every GPU folds them in the same rank order, from rank 0 through rank $N-1$. Transport remains asynchronous, but the floating-point expression is identical across replicas.

Sharded reductions obtain the same property through ownership. Each output block has one owner GPU, which performs the reduction once and distributes that result to the other GPUs. The receivers do not independently recompute the block, so their arrival order cannot create different copies. These two cases yield the design rule used throughout flash-comm: an output block is either computed once and distributed, or every independent replica evaluates its contributions in the same order. This rule guarantees agreement among the participants of one collective. The resulting value is defined by the common rank order or by the single owner's computation, as implemented by the kernels in Section 4.

4 Implementation

We implement flash-comm as a standalone communication backend for SGLang [1]. The Python runtime manages admission, symmetric storage, and policy dispatch; architecture-specific CUDA kernels implement the data paths in Section 3. Hopper and Blackwell use separate kernel trees and policy tables, while sharing the same communicator interface. The backend is constructed once for a process group and invoked directly from the serving runtime's collective selection chain.

4.1 Runtime Integration and Admission

The serving runtime constructs flash-comm for eligible process groups. During initialization, the communicator checks that the process group runs on CUDA, the GPU architecture has a policy table, the group lies inside a fully connected NVLink domain, the group size is 2, 4, or 8, and the configured data type is bf16 or fp32. It then allocates the shared storage described below and compiles the CUDA implementation for the detected compute-capability family.

Each invocation performs a second, tensor-level admission check. The tensor must use the communicator's device and data type, have a peer-accessible weakly contiguous layout, satisfy 16-byte address and length alignment, remain below the configured payload ceiling, and match an entry in the policy table for its group size. Eligible tensors enter flash-comm; all other tensors continue through the serving runtime's existing backend chain. This admission path integrates the optimized kernels without changing communication for other layouts, data types, or message sizes.

The runtime dispatch itself is deliberately simple. GPU compute capability selects the Hopper or Blackwell backend, and a monotone table indexed by group size and message size selects an algorithm identifier and fixed launch shape. The request path performs neither fitting nor autotuning. Tables are produced offline from the measurement procedure described in Section 3.2 and evaluated in Section 5.

4.2 Symmetric Storage and Graph-Safe Reuse

The communicator allocates one symmetric-memory slab per GPU. The allocation provides two views of the same storage: peer pointers for unicast loads and stores, and an NVSwitch multicast alias for `multimem.*` operations [2]. The slab is partitioned by algorithm family because one-hop publication, reduce-scatter publication, and all-gather results require different layouts and maximum strides. Each family receives only the capacity required by its largest shipping policy entry.

Initialization first zeroes switch-reduction scratch regions and fills the arrival-checked regions with the data type's sentinel. A group rendezvous and barrier then ensure that every multicast alias is live and no peer can publish into a region while another process is still initializing it. The communicator performs these setup barriers once during construction. Steady-state collective invocations begin with fully initialized aliases and storage.

Every family divides its storage into four rotating slots. Reusing a slot on successive operations would allow a fast rank to observe a value left by an earlier call, so each kernel derives its slot from a device-local call counter. The counter is maintained per warp and per algorithm family. Computing the slot on device is required for CUDA Graph execution: host launch arguments are captured once and cannot advance a call index during graph replay. The readiness encoding proves that a value belongs to the active protocol state; slot rotation provides temporal separation between successive invocations.

This layout imposes two invariants. First, all ranks must issue the same sequence of collective calls, including empty tensors, so their counters remain aligned. Second, an algorithm family uses one fixed grid. Changing the mapping between warps and counters would cause different threads to select different slots. Production dispatch therefore obtains the launch shape from the same policy entry that sized the counters and slab.

4.3 Communication Kernels

One-hop transfer. For smaller messages, each GPU publishes its input directly to the other participants through unicast peer stores or an NVSwitch multicast store. Receive storage is indexed by source rank. A thread polls the required source slots, resets each sentinel after capturing the value, and performs the reduction only after all contributions are available. For more than two GPUs, contributions are held by source rank and folded in rank order, as described in Section 3.3. A warp-per-peer variant assigns independent warps to sources, turning sequential peer waits into concurrent polling when readiness latency dominates.

The low-latency two-GPU variant replaces sentinels with sequence-numbered packets. Each 16-byte packet carries 8 bytes of payload and two copies of the current sequence number. The receiver accepts the packet when the sequence matches its call index, then reduces the payload without a sentinel reset. Because this encoding doubles the transferred bytes, the shipping policy uses it only through 128 KB on Blackwell with two GPUs.

Sharded two-hop transfer. For larger groups and messages, the output is divided into N shards. Every GPU first publishes its input in the reduce-scatter region. The owner of shard r obtains the reduction for that shard, either by reading through `multimem.ld_reduce` or by explicitly folding published peer values. It then distributes the completed shard through multicast or peer stores. Receivers poll the all-gather region and place each completed shard in the final output.

Table 3: The two-hop flash-comm kernel removes both cross-GPU bookend barriers. Barrier counts cover group-wide and semaphore-style inter-GPU coordination in the steady-state collective.

Implementation	Entry condition	Exit condition	Barriers
NCCL symmetric window	LSA multimem barrier	release synchronization	2
SGLang custom AR v2	`barrier.arrive_relaxed(0)`	`barrier.arrive_rel_acq(1)`	2
flash-comm	input/output readiness evidence	block consumption and rearming	0

Sentinel checks connect the two phases at block granularity, so an owner can distribute one completed block while other blocks are still waiting for input.

The kernel overlaps publication, reduction, and result collection through bounded windows. A window deeper than the number of packs owned by a thread degenerates into repeated dependency waits, so pipeline depth is capped by the available work. Publication is also split between the threads reducing the local shard and those publishing remote shards, avoiding duplicated ownership. These launch and pipeline parameters are encoded in the offline policy table rather than selected dynamically inside the kernel.

All-gather. The result-distribution half of the two-hop kernel already implements an all-gather: each source publishes one shard and receivers wait for the remote shards through the same readiness layer. The standalone all-gather reuses the symmetric slab, device-side slot rotation, and sentinel polling, omitting only the reduction step. Its performance is reported in Section 5.3.

4.4 Launch Dependencies and Progress

All shipping kernels use Programmatic Dependent Launch (PDL) [3] to connect to neighbouring GPU work under CUDA Graph execution. Slot selection, offsets, and shard indices do not depend on the input values, so flash-comm computes them before the PDL wait and overlaps this front-end work with completion of the preceding kernel. Payload reads occur only after the dependency is satisfied, and the kernel signals dependent work after its output is published.

Production readiness loops persist until the required peer data arrives. This requires publishing and polling blocks to remain concurrently resident, so the adapter admits launch grids within the measured co-residency bound. For a non-finite switch-reduction result, the kernel changes from aggregate polling to explicit source verification after a bounded interval. It completes as soon as every source is present, including inputs containing a legitimate NaN or infinity.

4.5 Steady-State Synchronization Cost

The two-hop path provides a source-level comparison with the strongest baselines because all three implementations use the same reduce-scatter plus all-gather structure. Their steady-state synchronization differs as follows. The two baseline bookends add separate cross-GPU coordination before and after data movement. flash-comm establishes both dependencies through block-level readiness evidence, removing the two phases from the steady-state path. This reduction in fixed protocol work produces the lower latency intercept measured in Section 5. Table 3 summarizes the resulting steady-state barrier counts.

5 Evaluation

We evaluate flash-comm at two levels. Collective microbenchmarks measure whether self-synchronizing transfers reduce the latency of all-reduce and all-gather across message sizes and GPU counts. We then integrate flash-comm into an LLM serving stack and measure whether these savings reduce time per output token (TPOT) during decoding.

5.1 Experimental Setup

Our collective experiments use one eight-GPU node and form communicators of two, four, and eight GPUs. We evaluate NVIDIA H100, H200, and B300 systems, which cover two NVLink generations. The evaluation uses two complementary views of all-reduce performance. A detailed eight-GPU B300 sweep shows how absolute latency changes with message size, while the full cross-system sweep shows how the median benefit changes with hardware and communicator width. The collective experiments cover message sizes from 1 KB to 16 MB. Unless stated otherwise, latency is measured inside CUDA Graph execution, where host launch overhead does not obscure the device-side synchronization cost.

For all-reduce, the primary comparison is SGLang custom AR v2, the strongest decode-oriented baseline over the target operating range. Both the detailed B300 comparison and the cross-system medians use this baseline. The all-gather experiment compares against the per-size fastest of three available baselines. All reported improvements use lower latency as positive.

The end-to-end experiment runs DeepSeek-V4-Flash-0731 with tensor-parallel decoding on all eight B300 GPUs. We compare the same serving workload using flash-comm and custom AR v2. TPOT measures the average latency

Table 4: Representative all-reduce latency on eight B300 GPUs. The reduction is relative to custom AR v2; latency is in microseconds.

Message size	Custom AR v2	flash-comm	Reduction
1 KB	3.49	3.33	4.6%
128 KB	4.92	4.74	3.7%
512 KB	9.45	6.58	30.4%
1 MB	12.36	7.08	**42.7%**
4 MB	20.70	14.78	28.6%
16 MB	53.43	42.48	20.5%

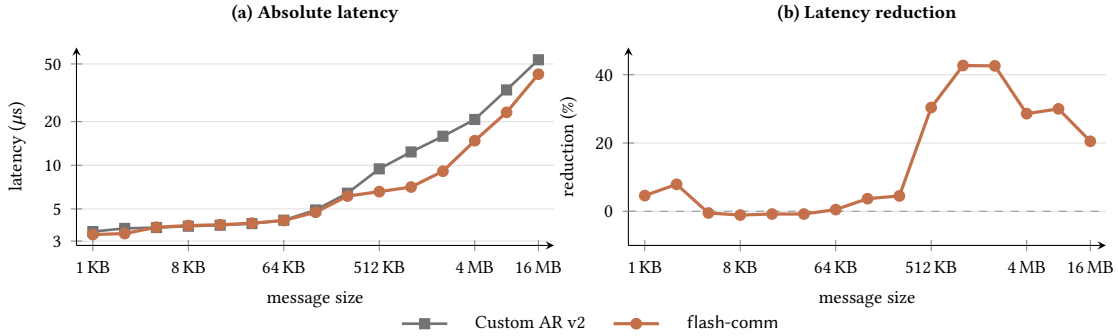


Figure 3: All-reduce latency on eight B300 GPUs. Panel (a) compares the absolute latency of custom AR v2 and flash-comm; panel (b) shows the corresponding reduction. Points sample 15 message sizes from the 188-size sweep.

Table 5: Representative all-gather latency on eight B300 GPUs. The baseline is the fastest available implementation at each message size; latency is in microseconds.

Message size	Fastest baseline	flash-comm	Reduction
1 KB	3.36	2.56	23.8%
128 KB	3.80	2.86	24.7%
512 KB	8.44	3.47	**58.9%**
1 MB	9.34	4.24	54.6%
4 MB	13.96	8.89	36.3%
16 MB	31.70	28.91	8.8%

between consecutively generated output tokens. We report concurrency 16 and above, which covers the loaded serving regime used by this experiment.

5.2 All-Reduce Performance

We first examine the detailed eight-GPU B300 sweep. Across all 188 message sizes, flash-comm reduces median latency by 28.8% and reaches a maximum reduction of 43.7%. Table 4 reports representative absolute latencies, while Figure 3 shows the size-dependent trend.

The largest gains occur once the selector moves to the sharded two-hop path, which reduces peer-scaled transfer volume while avoiding the two bookend barriers described in Section 4.5. The advantage reaches 43.7% over the complete sweep. At small messages, all implementations use low-overhead one-hop protocols and fixed kernel work dominates. Consequently, flash-comm is within approximately 1% of custom AR v2 at the weakest eight-GPU points, including a small 4–16 KB loss region.

The absolute latency comparison gives the same picture. On eight B300 GPUs, the implementations remain close through the small-message range. At 512 KB, custom AR v2 takes 9.45 μs while flash-comm takes 6.58 μs. At 16 MB, the corresponding latencies are 53.43 and 42.48 μs. Thus the median gain does not come from a uniform shift across all sizes. It comes primarily from the region where removing separate synchronization and reducing transferred bytes both matter.

5.3 All-Gather Performance

flash-comm implements both all-gather and all-reduce using the same self-synchronizing transfer design. The all-gather implementation uses symmetric storage, slot rotation, and block-level readiness checks to exchange each rank's shard among the participating GPUs. Table 5 reports representative absolute latencies on B300 against the fastest baseline at each tested size. flash-comm improves all 45 measured combinations of message size and GPU count. Across eight GPUs, it reduces latency by a median of 23.8% and by as much as 58.9%. Because all-gather exchanges shards without a reduction stage, its improvement comes directly from lowering fixed coordination cost and allowing each receiver to validate the blocks it needs. These results show that the same self-synchronizing transfer design benefits both all-gather and all-reduce.

Table 6: Median all-reduce latency reduction relative to custom AR v2 across 188 message sizes. The benefit grows with the number of participating GPUs.

System	2 GPUs	4 GPUs	8 GPUs
H100 (NVLink 4)	8.6%	11.8%	24.3%
H200 (NVLink 4)	4.9%	9.5%	22.3%
B300 (NVLink 5)	4.2%	9.8%	**28.8%**

Table 7: Flash-comm lowers end-to-end TPOT by 1.7–6.8% relative to custom AR v2 across all reported concurrency levels. TPOT is in milliseconds; lower is better.

Concurrency	Custom AR v2	flash-comm	Reduction vs. v2
16	2.99	**2.94**	1.7%
32	4.38	**4.08**	**6.8%**
48	5.45	**5.16**	5.3%
64	6.49	**6.18**	4.8%
96	8.32	**8.06**	3.1%
128	10.15	**9.88**	2.7%
192	13.60	**12.89**	5.2%
256	9.92	**9.70**	2.2%
384	9.94	**9.65**	2.9%
512	10.01	**9.76**	2.5%

5.4 Cross-Generation Calibration

The relative costs of unicast, multicast, and switch-assisted reduction vary across NVLink generations, which can shift the protocol switching boundaries. We therefore examine how the cost-guided policy adapts across hardware platforms. Appendix A lists the machine constants used for calibration.

Table 6 reports the median reduction over all 188 message sizes for each configuration. All three systems obtain their largest benefit at eight GPUs, where the reduction reaches 22.3–28.8%. The magnitude nevertheless differs across machines, including between H100 and H200 within the same NVLink generation.

The same protocol boundary therefore cannot be copied unchanged across GPU generations or neighbouring systems. On H100 and H200, multicast throughput is approximately one third of unicast throughput, while on B300 the two are comparable. These different slopes and fixed costs change which data path is efficient and where two fitted protocol costs cross. Accordingly, flash-comm uses architecture-specific policy tables calibrated from the cost model in Section 3.2. The selector uses the analytical crossing only to locate the search region and places the runtime boundary on a neighbouring measured point.

5.5 End-to-End Decoding

Table 7 reports TPOT from concurrency 16 to 512. flash-comm has the lowest TPOT at all ten reported points against custom AR v2. Relative to custom AR v2, it reduces TPOT by 1.7–6.8%. The largest relative gains occur at concurrency 32–64, where TPOT falls by 4.8–6.8%. The benefit remains positive at higher concurrency, including 5.2% at concurrency 192 and 2.2–2.9% from concurrency 256 to 512. The relative and absolute savings emphasize different operating points. At concurrency 32, flash-comm achieves the largest relative reduction, lowering TPOT from 4.38 to 4.08 ms. At concurrency 192, it achieves the largest absolute reduction, lowering TPOT from 13.60 to 12.89 ms, a saving of 0.71 ms per output token. The improvement is therefore not monotonic in either concurrency or baseline TPOT, consistent with the workload exercising different collective sizes and policy bands as concurrency changes. Because the model and serving configuration remain fixed across backends, the consistent lead across the full range shows that the collective-level improvements carry through to the decoding path under multiple load regimes.

6 Discussion and Limitations

6.1 Deployment Scope

flash-comm targets collective communication within a single high-bandwidth GPU domain. The current implementation supports communicators of two, four, and eight GPUs connected through NVLink and NVSwitch. This scope matches the single-node tensor-parallel groups commonly used during LLM decoding, where collectives recur at every layer and generated token. Extending the design across nodes would introduce network transports, routing, and failure handling with different readiness and cost structures.

The optimized kernels require Hopper or Blackwell GPUs and use CUDA features including symmetric peer-accessible memory, `multimem.*` operations, and Programmatic Dependent Launch. The runtime admits bf16 and fp32 tensors with peer-accessible layouts and 16-byte alignment. Other devices, data types, and layouts continue through

the serving runtime's existing communication backends. These requirements allow readiness evidence and payload data to share the same transfer path while preserving CUDA Graph execution.

6.2 Message Range

The current policy tables cover messages from 16 bytes through 16 MB. This range includes the small and medium collectives that dominate tensor-parallel decoding, from short control and partial-result transfers to multi-megabyte hidden-state exchanges at higher concurrency. The evaluation focuses on 1 KB–16 MB, where fixed synchronization cost and transferred bytes both materially affect latency. Messages outside the optimized policy range use the next communication backend selected by the serving runtime.

The supported range is also where cost-guided selection is most useful. For short messages, fixed readiness work dominates and compact sequence-number protocols can be effective. As messages grow, their additional bytes become more expensive, favouring sentinel readiness. At still larger sizes and wider GPU groups, sharded two-hop communication reduces peer-scaled traffic enough to repay its additional phase. A single protocol therefore cannot cover the decode range efficiently.

6.3 Recalibration across GPU Generations

The selection method transfers across GPU generations, while its calibrated boundaries remain hardware specific. The intercept and slope in Equation 1 depend on kernel launch cost, barrier latency, peer-access latency, and the relative throughput of unicast, multicast, and switch-assisted reduction. These quantities change at different rates as GPU and interconnect generations evolve. For example, multicast is substantially slower than unicast on the evaluated Hopper systems but reaches comparable throughput on B300. The same communication path consequently crosses its alternatives at a different message size.

flash-comm handles this variation through architecture-specific policy tables. For each platform, offline measurements fit the fixed and per-byte costs of the candidate protocols, locate their crossing regions, and benchmark neighbouring message sizes to select measured boundaries. Runtime dispatch then reduces to a table lookup. This separation preserves the main design principle across hardware generations: the cost model and calibration procedure remain stable, while the selected readiness encoding, data path, and switching points adapt to the target system.

7 Related Work

7.1 Low-Latency GPU Collectives

NCCL provides architecture-tuned collective algorithms and protocols across a wide range of topologies and message sizes [4]. Prior analyses have exposed how its protocol, channel, and algorithm choices determine latency and bandwidth [5]. More recent systems push small-message collectives closer to the hardware limit. Every Microsecond Matters specializes GPU collectives for latency-sensitive workloads and carefully accounts for kernel and transport overheads [6]. SiFAR targets inference all-reduce and removes global synchronization through a synchronization-free data path on H200 [7]. Studies of the synchronization tax further quantify how coordination limits GPU scale-up performance across workloads and hardware generations [8].

flash-comm shares the goal of minimizing exposed collective latency, but focuses on how data readiness is represented and consumed. Its self-synchronizing transfers attach readiness evidence to each block, allowing the receiver to advance at block granularity and re-arm storage through the same protocol. The cost-guided selector then chooses among readiness encodings and communication paths according to their fixed and per-byte costs. This combination extends the mechanism from all-reduce to all-gather and makes protocol boundaries an explicit, hardware-calibrated decision.

Asynchronous arrival also creates a numerical concern that is separate from ordinary approximation error. Floating-point addition is not associative, so replicas that reduce contributions in arrival order can diverge even when they receive identical inputs [9]. flash-comm preserves cross-GPU agreement by assigning a common source-rank order to replicated reductions and a single owner to each sharded output block. Transport can therefore remain asynchronous without making the result depend on arrival timing.

7.2 Communication Optimization for LLM Systems

Another line of work reduces exposed communication by overlapping it with computation. TokenWeave schedules distributed inference to increase compute-communication overlap [10]. Comet and tile-level signaling systems coordinate communication and computation at fine granularity for mixture-of-experts workloads [11, 12], while Domino applies tensor slicing and overlap to distributed transformer execution [13]. X-Stage models the backpressure between communication issue and remote visibility to improve fusion decisions [14]. These systems change when communication executes or how much of it is visible on the critical path.

flash-comm instead reduces the latency of the collective operation itself. This is particularly important during decoding, where a dependent layer often waits for the collective result and the short per-token computation window limits the amount of communication that can be hidden. The two approaches are complementary: scheduling and fusion

can overlap the portion permitted by the dependency graph, while self-synchronizing transfers reduce the remaining transport and coordination cost.

7.3 Scaling and Hardware-Aware Selection

Large-scale collective systems optimize routing, topology, and bandwidth for multi-node deployments containing thousands of GPUs [15, 16]. Their dominant costs differ from the single-node decode regime studied here, where repeated small and medium transfers make fixed synchronization latency visible. flash-comm therefore specializes within one NVLink domain and leaves inter-node routing and transport to the existing communication stack.

Hardware-aware algorithm selection is already fundamental to collective libraries. Our measurements show that readiness protocols require the same treatment: barrier latency and the relative throughput of unicast, multicast, and switch-assisted reduction change across GPU generations. flash-comm captures these changes through the intercept and slope of its cost model, then calibrates the switching points with neighbouring measurements. The portable artifact is the selection method, while the resulting policy table is tied to the target GPU and interconnect generation.

8 Conclusion

Collective communication remains on the critical path of distributed LLM decoding, even on modern high-bandwidth GPU systems. Our measurements identify separate readiness synchronization as an important source of this cost.

flash-comm removes these phases through self-synchronizing transfers that fuse readiness synchronization with data movement. Each block carries evidence of its own readiness, allowing receivers to validate, consume, and safely reuse storage without a separate group-wide barrier. A cost-guided selector balances the fixed work of readiness handling against additional transferred bytes and chooses among readiness encodings, one-hop exchange, and sharded two-hop communication. Finally, flash-comm decouples transport order from reduction order, preserving cross-GPU agreement while allowing blocks to arrive asynchronously.

On eight B300 GPUs, flash-comm reduces all-reduce latency by a median of 28.8% and up to 43.7% relative to custom AR v2. The same mechanism reduces all-gather latency by a median of 23.8% and up to 58.9%. In end-to-end decoding, these collective improvements reduce TPOT by 1.7–6.8%. The principle also applies to all-to-all, which has similar peer-readiness dependencies without a reduction step. We plan to adapt flash-comm to all-to-all and evaluate its impact on expert-parallel workloads. More importantly, our extensive experiments demonstrate that the benefits of this design generalize across a wide range of message sizes and GPU generations. Together, these results provide a practical path for keeping decode communication efficient as GPU systems continue to evolve.

References

[1] LMSYS Org. SGLang: A fast serving framework for large language models and vision language models. `https://github.com/sgl-project/sglang`, 2026.

[2] NVIDIA Corporation. *Parallel Thread Execution ISA*. NVIDIA Corporation, 9.0 edition, 2026. Documentation for `multimem` instructions.

[3] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA Corporation, 13.0 edition, 2026. Programmatic Dependent Launch and synchronization.

[4] NVIDIA Corporation. NVIDIA Collective Communications Library (NCCL). `https://github.com/NVIDIA/nccl`, 2026.

[5] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, Cedell Alexander, Eric Spada, James Dinan, Jeff Hammond, and Torsten Hoefler. Demystifying NCCL: An in-depth analysis of GPU communication protocols and algorithms. *arXiv preprint arXiv:2507.04786*, 2025.

[6] Siyuan Shen, Anton Korzh, John Bachan, Tiancheng Chen, Arnav Goel, Ludwig Schneider, Pouya Kousha, Zhenhao He, Sylvain Jeaugey, Kamil Iskra, Nishank Chandawala, Jeff R. Hammond, and Torsten Hoefler. Every microsecond matters: Achieving near-speed-of-light latency in GPU collectives. *arXiv preprint arXiv:2607.16100*, 2026.

[7] Hritvik Taneja, Anish Saxena, Abhishek Revinipati, Jae Hyung Ju, Neal C. Crago, and Moinuddin Qureshi. SiFAR: Synchronization-free all-reduce for low-latency LLM inference. *arXiv preprint arXiv:2607.08973*, 2026.

[8] Arjun Devraj, Lindsey Bowen, and Rachee Singh. Understanding the synchronization tax in GPU scale-up domains. *arXiv preprint arXiv:2608.22503*, 2026.

[9] IEEE Computer Society. IEEE Standard for Floating-Point Arithmetic, 2019.

[10] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. TokenWeave: Efficient compute-communication overlap for distributed LLM inference. *arXiv preprint arXiv:2505.11329*, 2025.

[11] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen, and Xin Liu. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. *arXiv preprint arXiv:2502.19811*, 2025.

[12] Minyu Cui, Anna Wingkvist, and Morgan Ericsson. Fine-grained computation-communication overlap via tile-level signaling and scheduling for mixture-of-experts. *arXiv preprint arXiv:2607.19539*, 2026.

Table 8: Synchronization and data-path costs change differently across GPU generations. Latency probes run inside CUDA Graphs; throughput is reported as effective GB/s.

Quantity	H100	H200	B300	Unit
Fixed and latency costs				
Empty-kernel floor	0.790	0.507	0.525	μs
Net cross-GPU barrier	2.66	1.87	4.33	μs
1 KB all-reduce, 8 GPUs	2.54	2.46	3.33	μs
Barrier / 1 KB all-reduce	1.05	0.76	1.30	ratio
L2 cross-side latency gap	44.6	44.0	93.6	ns
Data-path throughput				
Unicast store, fixed shape @256 MB	287.3	322.6	261.1	GB/s
Multicast store, fixed shape @256 MB	98.4	98.3	260.8	GB/s
Unicast store, shape-swept ceiling	359.9	359.3	652.6	GB/s
Multicast team egress, 8 GPUs	363.2	363.2	709.3	GB/s
All-to-all egress, 8 GPUs	334.4	334.8	636.2	GB/s
Peer `red.add`	324.4	324.8	558.8	GB/s
Switch `multimem.red`	98.4	98.1	237.7	GB/s
Local reduction	840.8	1128.7	2337.9	GB/s
Local HBM copy	2712.1	3659.3	6634.8	GB/s

[13] Guanhua Wang, Chengming Zhang, Zheyu Shen, Ang Li, and Olatunji Ruwase. Domino: Eliminating communication in LLM training via generic tensor slicing and overlapping. *arXiv preprint arXiv:2409.15241*, 2024.

[14] Jianwen Xian, Zhiyuan Xu, Yuchen Li, Ziliang Lai, Kang He, Zhen Huang, Aichen Feng, Jinyan Chen, Yilin Zhang, Qinqin Chen, Julien Lai, and Chengru Song. X-Stage: Modeling post-issue backpressure in GPU communication–computation fusion. *arXiv preprint arXiv:2607.23264*, 2026.

[15] Min Si, Pavan Balaji, Yongzhou Chen, Ching-Hsiang Chu, Adi Gangidi, Saif Hasan, Subodh Iyengar, Dan Johnson, Bingzhe Liu, Regina Ren, Deep Shah, Ashmitha Jeevaraj Shetty, Greg Steinbrecher, Yulun Wang, Bruce Wu, Xinfeng Xie, Jingyi Yang, Mingran Yang, Kenny Yu, Minlan Yu, Cen Zhao, et al. Collective communication for 100k+ GPUs. *arXiv preprint arXiv:2510.20171*, 2025.

[16] Prajwal Singhania, Siddharth Singh, Lannie Dalton Hough, Akarsh Srivastava, Harshitha Menon, Charles Fredrick Jekel, and Abhinav Bhatele. Understanding and improving communication performance in multi-node LLM inference. *arXiv preprint arXiv:2511.09557*, 2025.

Appendix A Platform Calibration Measurements

Table 8 reports the measurements used to calibrate Equation 1. Latency quantities contribute primarily to the fixed term a_p, while data-path throughput determines the per-byte term b_p. All systems use the same probe definitions, and latency probes execute inside CUDA Graphs. Fixed-shape throughput rows use an identical launch shape across systems; ceiling rows sweep launch shapes and report the best observed result.

Two cross-generation changes directly affect protocol selection. First, a standalone barrier is 0.76× the latency of a 1 KB all-reduce on H200, but rises to 1.30× on B300. Second, under the fixed-shape probe, multicast store throughput is 0.34× and 0.30× unicast on H100 and H200, respectively, but reaches 1.00× on B300. Blackwell therefore changes both the value of removing a synchronization phase and the relative cost of multicast publication.

The systems also differ in the resources surrounding the fabric. H100 and H200 provide 132 SMs and 50.0/60.0 MiB of L2, while B300 provides 148 SMs and 126.5 MiB of L2. The eight-GPU reduce-scatter/all-gather overlap probe reaches 68% on H100 and H200 and 52% on B300; the peer-write probe observes no torn 128-byte writes on any system. flash-comm combines these measured fixed and per-byte costs to locate each protocol crossing, benchmarks the neighbouring message sizes, and installs the resulting boundary in the architecture-specific policy table.